

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING



PROJECT REPORT
Library Management System

Name of students: Phạm Minh Tuyên – 20233885
 Lê Hà Hải Vân – 20233886
 Bùi Thanh Thảo – 20233876
 Nguyễn Thị Thu Hà – 20233848

Name of instructors: Dr. rer. nat. Nguyễn Thanh Bình

Hanoi, 06/2026

Table of Contents

1. Introduction & Project Overview	4
1.1 Background	4
1.2 Project Objectives	4
1.3 Technology Stack.....	5
1.4 Summary of Changes in This Version	5
2. Requirements Analysis.....	7
2.1 Stakeholders	7
2.2 Functional Requirements.....	7
2.3 Non-Functional Requirements	10
3. Business Process Specification	11
3.1 Revised Borrowing Process (Self-Service)	11
3.2 Borrowing Rules by Reader Type.....	12
3.3 Validation Rules (RuleEngine.validate).....	13
4. System Architecture	13
4.1 Architectural Pattern — MVC	13
4.2 Security Architecture (Role Separation)	14
4.3 Session Management.....	15
4.4 Transaction Management	15
4.5 Scheduling.....	15
4.6 Request Flow — Self-Service Submit & Pickup	15
5. Database Design.....	16
5.1 Entity-Relationship Overview	16
5.2 Table Definitions.....	16
6. Feature Implementation.....	21
6.1 Feature List.....	21
6.2 Key Business Logic Components	24
6.3 Changes & Fixes Applied in This Version.....	26
7. Task Assignment.....	28
8. Installation & Setup Guide.....	29
8.1 Prerequisites	29

8.2 Run in Development (H2).....	29
8.3 Sample Accounts (password: password123).....	30
8.4 How to Test the New Flow	31
9. Testing & Validation.....	31
9.1 Functional Test Scenarios (verified)	31
9.2 Build & Runtime Verification.....	33
10. Business Process: Flowchart & Walkthrough	33
10.1 Original Process — Phenikaa University Library (QT.03.TV.DV).....	34
10.2 Our System — Digital Self-Service Process	35
10.3 Comparison — Manual Process vs. Our System	39
11. System Design Models (UML)	40
11.1 Use Case Diagram.....	40
11.2 Class Diagram — Domain Model.....	41
11.3 Activity Diagram — Self-Service Borrowing.....	42
11.4 State Machine Diagram — Loan Lifecycle.....	43
12. Conclusion & Future Work.....	44
12.1 Achievements.....	44
12.2 Limitations	45
12.3 Future Development Roadmap.....	45

1. Introduction & Project Overview

1.1 Background

Phenikaa University Library manages the borrowing and returning of academic documents for thousands of students, lecturers, and researchers every semester. The traditional paper-based workflow — physical card presentation, handwritten loan slips, and manual ledger entries — is time-consuming and error-prone.

The first version of this project digitised that workflow but still imitated the old counter model: every loan request had to be manually approved by a librarian before the reader could borrow. Following the supervisor's review, the system has been redesigned to be genuinely more convenient than traditional borrowing:

- No librarian/admin approval is required. If the reader satisfies the borrowing rules, the loan is granted immediately.
- Each loan request generates a unique pickup code. Librarians simply check the books for that code.
- The reader picks up the books within 24 hours by visiting the library; the librarian enters the pickup code to confirm handover; otherwise the request auto-cancels and the held copies are released.
- The roles of Administrator and Librarian are now clearly separated. The administrator manages user accounts and a functional-permission matrix (which actions each role may perform), while the librarian configures the borrowing rules and shows/hides document types per role from a configuration screen — all without editing any code.

1.2 Project Objectives

- Provide a self-service, cart-based borrowing experience with no manual approval step.
- Issue a pickup code per request, hold the copies, and auto-cancel un-collected requests after 24 hours.
- Let the librarian confirm handover by entering the reader's pickup code at `POST /librarian/loans/confirm-pickup` — no manual paper ledger entry required.
- Enforce role-based borrowing rules (quota, duration, renewals, late fines) for Students, Lecturers, and Researchers.

- Separate the Administrator (user accounts + functional-permission matrix) from the Librarian (day-to-day operations + borrowing-rule configuration).
- Offer on-screen configuration with no code change and no redeploy: the librarian edits borrowing rules and document-type visibility per role, and the administrator grants or revokes functional permissions per role.

1.3 Technology Stack

Layer	Technology	Version / Notes
Backend Framework	Spring Boot	3.4.13 — Java 21, Maven
Web MVC	Spring Web MVC + Thymeleaf	Server-side template rendering
Security	Spring Security	Form login, strict role-based access (no role hierarchy)
Scheduling	Spring Scheduling (@EnableScheduling)	Background job auto-cancels expired pickups
ORM / Persistence	Spring Data JPA / Hibernate	H2 in-memory (dev), MySQL-compatible
Database (Dev)	H2 In-Memory	Auto-seeded via data.sql on startup
Database (Prod)	MySQL	Switch via application.properties
UI Framework	Bootstrap 5.3.2 + Bootstrap Icons 1.11.3	Responsive layout
Build Tool	Maven 3.8+	pom.xml dependency management
Language	Java 21	Records, switch expressions, pattern matching

1.4 Summary of Changes in This Version

This revised edition reflects every change requested in the supervisor's feedback. The table below summarises the difference between the first submission and the current version.

Aspect	(previous)	(current)
Approval	Librarian must approve each PENDING loan	No approval — granted immediately if eligible
Pickup	Implicit, no code	Pickup code + 24h window; librarian enters code to confirm handover
Un-collected loans	Stay forever / manual cancel	Auto-cancelled after 24h by a scheduled job
Roles	ADMIN + readers only	ADMIN, LIBRARIAN + readers (clearly separated)
Admin scope	Everything (loans, books, users)	Only Users + a functional-permission matrix (no loans/books/rules)
Librarian scope	(did not exist)	Dashboard, pickup orders, returns, book/stock management
Doc-type visibility	Hard-coded	Toggled per role by the librarian on the /librarian/rules page
Loan status	PENDING/APPROVED/RETURNED/CANCELLED	AWAITING_PICKUP / BORROWED / COMPLETED / CANCELLED
Spring Boot	3.3.5	3.4.13
Notifications	(none)	In-app notifications for loan / pickup / return / fine / cancel events (navbar bell)
In-house reading	(none)	Librarian issues read-on-site slips for restricted documents
Fine settlement	(none)	Per-item fine tracked & marked paid; a loan completes only when settled

2. Requirements Analysis

2.1 Stakeholders

Stakeholder	Role in System	Key Needs
Student	Borrower (STUDENT)	Search, request loans, show pickup code at library counter, view history, renew
Lecturer / Staff	Borrower (LECTURER)	Unlimited loan duration on textbooks & specialised references
Researcher	Borrower (RESEARCHER)	Extended quotas on specialised references
Librarian	Operations (LIBRARIAN)	Confirm handover for pickup orders (enter pickup code at counter), process returns and fines, configure borrowing rules and document-type visibility
Administrator	System admin (ADMIN)	Manage users & roles; configure the functional-permission matrix per role

2.2 Functional Requirements

FR-1 Authentication & Routing

- Authenticate users by student/staff code and BCrypt password.
- Deny borrowing to accounts whose card status is LOCKED.
- After login, route ADMIN to user management, LIBRARIAN to the operations dashboard, and readers to the client home page.

FR-2 Catalogue & Search

- Display books with title, author, ISBN, document type, classification code, and available copies.
- Search by keyword (title / author / ISBN / classification code) and filter by document type and availability.
- Show a reader only the document types that are visible for their role (configurable by the librarian).

FR-3 Cart & Self-Service Loan Request

- A reader can add up to 5 books to a session cart.
- On submit, the RuleEngine validates the cart; if valid the loan is created immediately (no approval).
- The loan is created with status `AWAITING_PICKUP`, a unique pickup code, and a 24-hour pickup deadline; held copies are decremented at once.

FR-4 Librarian Confirms Pickup via Code

- The reader visits the library within the 24-hour window and shows the pickup code to the librarian.
- The librarian enters the code at `POST /librarian/loans/confirm-pickup`; the system locates the matching `AWAITING_PICKUP` loan by code.
- On success the loan becomes `BORROWED` and each item's due date is set per the reader's role and document type.

FR-5 Auto-Cancellation of Expired Pickups

- A scheduled job runs periodically and cancels every `AWAITING_PICKUP` loan whose 24-hour deadline has passed.
- Cancellation restores the held copies back to the catalogue.

FR-6 Book Return & Fine Calculation

- A librarian processes the return of each loan item.
- The overdue fine is computed at 2,000 VND per overdue day (full day count).

- The returned condition (GOOD / DAMAGED) is recorded; the parent loan auto-closes (status COMPLETED) once all items are returned and all fines are settled.

FR-7 Renewal

- A reader can renew an active item within the permitted number of renewals.
- A renewal extends the due date by the configured `renewal_days`.
- Renewal is denied for items not owned by the reader, not in BORROWING status, or beyond the renewal limit.

FR-8 Administrator — Users & Permissions

- Create, update, search and delete user accounts (including LIBRARIAN) and assign roles from the UI; lock/unlock reader cards and reset passwords.
- Grant or revoke functional permissions per role on the `/admin/permissions` page (book CRUD, return, pay-fine, cancel, in-house lending, rule management, borrow / renew / self-cancel).
- The administrator's own management permissions (users + permission matrix) are fixed and cannot be revoked; the admin does not touch books, loans or borrowing rules.

FR-9 Librarian — Operations

- View pickup orders (codes and deadlines) to see expected arrivals; confirm handover by entering pickup code when reader arrives at counter.
- Enter the reader's pickup code at `POST /librarian/loans/confirm-pickup` to confirm handover — the loan transitions to BORROWED and due dates are set per role × document type.
- Process returns, record condition and collect overdue fines; manage the book catalogue and copy counts (CRUD).
- View the operations dashboard (awaiting pickup, active borrowings, overdue items).
- Edit the borrowing-rule matrix and toggle document-type visibility per role on the `/librarian/rules` page — without modifying code.

- Issue in-house reading slips for restricted (internal) documents on the /librarian/inhouse page.
- Cancel a pickup order with a mandatory reason that is sent to the reader; the held copies are released back to stock.

FR-10 Account Self-Service (seeded accounts & password change)

- The database is pre-seeded with 82 accounts (2 administrators and 20 each of student, lecturer, researcher and librarian), all using the default password password123.
- After signing in, any user can change their own password at /account/password; the current password is verified and the new one (≥ 6 characters) is stored as a BCrypt hash.

FR-11 In-App Notifications

- Business events — loan created, pickup confirmed, return, fine collected, cancellation and completion — raise in-app notifications shown through a navbar bell.
- A reader is notified about their own orders; every librarian is notified about new orders, pickup confirmations and auto-cancellations (NotificationService.notify / notifyRole).
- Users open /notifications to read the full list and mark items read individually or all at once.

2.3 Non-Functional Requirements

Category	Requirement
Security	Strict RBAC. /admin/** requires ADMIN; /librarian/** requires LIBRARIAN; /client/** requires a reader role; /notifications/** requires any authenticated user. No role inheritance between ADMIN and LIBRARIAN.
Data Integrity	All mutating service methods are @Transactional; copies are validated for all cart items before any decrement.
Reliability	A scheduled job guarantees that held copies are released if a reader never collects the books.

Usability	Bootstrap 5 responsive UI; inline validation messages; pickup code shown prominently to the reader.
Portability	H2 in-memory for zero-config development; MySQL-compatible schema for production.
Maintainability	Clean MVC layering; all borrowing calculations encapsulated in RuleEngine; rule administration in RuleService.

3. Business Process Specification

3.1 Revised Borrowing Process (Self-Service)

The counter-approval step of the original 8-step process has been removed. The digital flow is now:

Step	Activity	Actor	Digital Implementation
1	Sign in	Reader	Login + card_status == ACTIVE check
2	Search documents	Reader	GET /client/home — keyword + type filter, role-based visibility
3	Build cart & submit	Reader	POST /client/loans/submit — RuleEngine validates
4	Loan granted instantly	System	Create loan AWAITING_PICKUP + pickup code + 24h deadline; hold copies
5	Collect books (self-service)	Reader	Reader visits library within 24 h and retrieves reserved books from open shelves; no system action required
6	Collect books; librarian confirms via code	Reader + Librarian	Reader presents pickup code at counter; librarian enters at POST /librarian/loans/confirm-pickup → BORROWED + due dates set per role × doc-type
7	Auto-cancel if not collected	System	Scheduled job cancels expired AWAITING_PICKUP loans, releases copies

8	Return / renew / fine	Reader + Librarian	Return via /librarian/loans/details/{id}/return; renew via /client/loans/renew/{detailId}
----------	-----------------------	--------------------	--

3.2 Borrowing Rules by Reader Type

These 12 rows seed the borrowing_rules table and are enforced by the RuleEngine. They match the QT.03.TV.DV regulation and are now editable by the librarian on the /librarian/rules page (including a per-row visibility switch).

Reader	Document Type	Max Qty	Loan Duration	Max Renewals	Days / Renewal
Student	Textbook	10	150 days	0	—
Student	Specialised Ref.	5	90 days	2	3 days
Student	General Ref.	3	15 days	0	—
Student	Restricted	0	—	0	Not allowed
Lecturer	Textbook	10	Unlimited	0	—
Lecturer	Specialised Ref.	5	Unlimited	0	—
Lecturer	General Ref.	5	15 days	2	3 days
Lecturer	Restricted	0	—	0	Not allowed
Researcher	Textbook	5	150 days	0	—
Researcher	Specialised Ref.	10	30 days	2	3 days
Researcher	General Ref.	10	15 days	0	—
Researcher	Restricted	0	—	0	Not allowed

3.3 Validation Rules (RuleEngine.validate)

Before a loan is created, the cart is checked in layers. Actual user-facing messages are in Vietnamese.

Layer	Rule	Message (shown in Vietnamese in-app)
Account	Card status must be ACTIVE	Your account is locked. Please contact the librarian.
Account	No outstanding overdue items	You have N overdue documents. Please return them before borrowing more.
Cart	Cart not empty	Your cart is empty. Please add books before submitting.
Cart	At most 5 items per loan	A single loan may not exceed 5 documents in total...
Type	Restricted documents not borrowable	Internal documents (red sticker) are read-on-site only and cannot be taken away.
Type	Type must be visible & permitted (max_quantity>0)	You are not allowed to borrow documents of type: [type name].
Quota	Per-type count $\leq$ max_quantity	Over the limit for type '[type]': max N, you selected M.

4. System Architecture

4.1 Architectural Pattern — MVC

Layer	Package	Key Classes
Presentation	controller/	ClientController, LibrarianController, AdminController, AuthController, AccountController, NotificationController
Business Logic	service/	LoanService, BookService, UserService, RuleService, NotificationService,

		PermissionService, CustomUserDetailsService
Rule Engine	util/	RuleEngine
Scheduling	config/	PickupExpiryScheduler
Data Access	repository/	LoanRepository, LoanDetailRepository, BookRepository, BorrowingRuleRepository, UserRepository, NotificationRepository, RolePermissionRepository
Domain Model	entity/	Loan, LoanDetail, Book, User, BorrowingRule, RolePermission, Permission, Notification (+ enums)
DTOs	dto/	CartItem, BookForm, ValidationResult, RuleConfigForm, ReturnBookForm
View	templates/	client/, librarian/, admin/, auth/, account/, fragments/, error/, notifications.html

4.2 Security Architecture (Role Separation)

Spring Security uses form login backed by CustomUserDetailsService (lookup by student_code, BCrypt passwords). Authorisation is strict — there is no role hierarchy, so ADMIN and LIBRARIAN are disjoint:

```
.requestMatchers("/login", "/css/**", "/images/**", "/error").permitAll()
.requestMatchers("/h2-console/**").permitAll()
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/librarian/**").hasRole("LIBRARIAN")
.requestMatchers("/client/**").hasAnyRole("STUDENT","LECTURER","RESEA
RCHER")
.requestMatchers("/notifications/**").authenticated()
.anyRequest().authenticated()
```

Post-login routing: ADMIN → /admin/users, LIBRARIAN → /librarian/dashboard, readers → /client/home.

The /notifications/** path needs only an authenticated user, and /account/password falls through to `.anyRequest().authenticated()`, so every role can read its notifications and change its own password.

4.3 Session Management

The cart is stored in the HTTP session as a `List<CartItem>` (60-minute timeout) and cleared on a successful submit.

4.4 Transaction Management

All mutating service methods are `@Transactional`. `submitLoan` first verifies availability for every cart item, then atomically creates the Loan and its `LoanDetails` and decrements available copies, so a single out-of-stock item never leaves partially-held copies behind.

4.5 Scheduling

`PickupExpiryScheduler` is a `@Component` whose `@Scheduled(fixedRate = 600000)` method (every 10 minutes) calls `LoanService.cancelExpiredPickups()`, which cancels `AWAITING_PICKUP` loans past their deadline and restores the held copies. `@EnableScheduling` is declared on the application class.

4.6 Request Flow — Self-Service Submit & Pickup

Browser → POST /client/loans/submit

→ `ClientController.submitLoan()`

→ `LoanService.submitLoan(user, cart)`

├─ `RuleEngine.validate(user, cart)` [account/cart/type/quota]

├─ verify all books available

├─ save Loan [AWAITING_PICKUP] + pickupCode + deadline(now+24h)

└─ for each book: save `LoanDetail` [RESERVED], `available_copies -= 1`

→ flash 'Pickup code: XXXXXX' → redirect /client/my-loans

Browser (Librarian) → POST /librarian/loans/confirm-pickup (pickup code entered by librarian)

→ `LibrarianController.confirmPickup(code)`

→ `LoanService.confirmPickupByLibrarian(code)`
 └─ find `AWAITING_PICKUP` loan by pickup code
 └─ `Loan` → `BORROWED`; each detail → `BORROWING` + `due_date` set;
reader notified

5. Database Design

5.1 Entity-Relationship Overview

users —< loans —< loan_details >— books

borrowing_rules (user_role + doc_type + visible)

role_permissions (user_role + permission) notifications (user_id, type, is_read)

loan_renewals (loan_detail_id, renewed_at, new_due_date)

- One User has many Loans (1–N).
- One Loan has many LoanDetails (1–N, cascade ALL).
- One LoanDetail references exactly one Book.
- BorrowingRule is keyed on (user_role, doc_type) and drives quota, due-date and visibility logic.
- RolePermission is keyed on (user_role, permission); Notification belongs to one User and backs the in-app alert feed; LoanRenewal records each renewal action (timestamp and new due date) for a LoanDetail.

5.2 Table Definitions

Table: users

Column	Type	Notes
id	BIGINT	PK, auto-increment
student_code	VARCHAR	UNIQUE, NOT NULL — login id
full_name	VARCHAR	NOT NULL
password_hash	VARCHAR	NOT NULL — BCrypt
role	VARCHAR(20)	STUDENT LECTURER RESEARCHER LIBRARIAN ADMIN

card_status	VARCHAR(10)	ACTIVE LOCKED
created_at	DATETIME	Account creation timestamp

Table: books

Column	Type	Notes
id	BIGINT	PK
title / author	VARCHAR	NOT NULL
isbn	VARCHAR	ISBN / ISSN
category	VARCHAR	Subject category
doc_type	VARCHAR(20)	TEXTBOOK SPECIALIZED_REF GENERAL_REF RESTRICTED
total_copies / available_copies	INT	Stock counters
classification_code, publisher, publish_year, description, image_url	—	Catalogue metadata

Table: loans

Column	Type	Notes
id	BIGINT	PK
user_id	BIGINT	FK → users.id
created_at	DATETIME	When the request was submitted
pickup_code	VARCHAR(10)	6-char code the reader uses to collect & confirm
pickup_deadline	DATETIME	created_at + 24h; after this the loan auto-cancels
picked_up_at	DATETIME	When the reader

		confirmed receipt
status	VARCHAR(20)	AWAITING_PICKUP BORROWED COMPLETED CANCELLED
in_house	BOOLEAN	TRUE for in-house (read-on-site) slips of restricted documents
cancel_reason	VARCHAR(255)	Reason shown to the reader when an order is cancelled
cancelled_by_role	VARCHAR(20)	Role that cancelled (LIBRARIAN / reader); NULL if auto- cancelled

Table: loan_details

Column	Type	Notes
id	BIGINT	PK
loan_id	BIGINT	FK → loans.id
book_id	BIGINT	FK → books.id
due_date	DATE	Set on pickup (NULL before pickup, or unlimited for lecturers)
return_date	DATETIME	Actual return timestamp
renewal_count	INT	DEFAULT 0
condition_status	VARCHAR(10)	GOOD DAMAGED (set on return)
fine_amount	DECIMAL	Overdue fine in VND
fine_paid	BOOLEAN	Whether the overdue fine has been collected (auto-true when fine = 0)
status	VARCHAR(20)	RESERVED BORROWING RETURNED

		OVERDUE
--	--	---------

Table: borrowing_rules

Column	Type	Notes
id	BIGINT	PK
user_role	VARCHAR	part of UNIQUE(user_role, doc_type)
doc_type	VARCHAR	part of UNIQUE(user_role, doc_type)
max_quantity	INT	Max copies per loan (0 = not permitted)
borrow_days	INT (nullable)	Loan duration; NULL = unlimited
max_renewals	INT	0 = no renewal
renewal_days	INT (nullable)	Days added per renewal
visible	BOOLEAN	NEW — whether this role sees/borrows this doc type (admin- toggled)

Table: loan_renewals

Column	Type	Notes
id	BIGINT	PK, auto-increment
loan_detail_id	BIGINT	FK -> loan_details.id
renewed_at	DATETIME	Timestamp when the renewal was performed
new_due_date	DATE	Extended due date after this renewal

Table: notifications

Column	Type	Notes
id	BIGINT	PK, auto-increment
user_id	BIGINT	FK -> users.id
type	VARCHAR(10)	SUCCESS INFO WARNING DANGER
message	VARCHAR(500)	Notification body text
link	VARCHAR(255)	Target URL for one-click navigation (nullable)
is_read	BOOLEAN	Whether the user has read this notification
created_at	DATETIME	When the notification was raised

Table: role_permissions

Column	Type	Notes
id	BIGINT	PK, auto-increment
user_role	VARCHAR(20)	Part of UNIQUE(user_role, permission)
permission	VARCHAR(50)	Part of UNIQUE(user_role, permission)
granted	BOOLEAN	TRUE = this role may perform the action

6. Feature Implementation

6.1 Feature List

Reader (client) features

#	Feature	Route	Description
1	Browse & Search	GET /client/home	Keyword/type/availability filters; only role-visible document types
2	Book Detail	GET /client/books/{id}	Full metadata, availability, in-cart indicator
3	Add to Cart	POST /client/cart/add (+ /client/cart/add-ajax)	Duplicate & 5-item limit, availability and visibility checks
4	View / Remove / Clear Cart	GET /client/cart, POST .../remove, .../clear	Manage session cart
5	Submit Loan (self-service)	POST /client/loans/submit	Validate → create AWAITING_PICKUP + pickup code; show code
6	My Loans	GET /client/my-loans	Awaiting-pickup orders (with code & countdown), active loans, history
7	Pickup Code (Show to Librarian)	GET /client/my-loans	Reader's 6-char pickup code visible on My Loans; reader brings it to the library counter for the librarian to enter at /librarian/loans/confirm-pickup.
8	Renew	POST /client/loans/renew/{detailId}	Ownership + BORROWING + renewal-limit checks
9	Change Password (all roles)	GET/POST /account/password	Verify current password, set a new one (≥ 6 chars), saved as BCrypt hash
10	Cancel Pickup	POST	Reader cancels an own

	Order	/client/loans/{id}/cancel	AWAITING_PICKUP order; held copies released
11	Notifications	GET /notifications, POST /notifications/{id}/read, /notifications/read-all	Read in-app notifications; mark one or all as read

Librarian features (operations)

#	Feature	Route	Description
12	Operations Dashboard	GET /librarian/dashboard	Awaiting-pickup count, active borrowings, overdue list
13	Pickup Orders & Confirm Handover	GET /librarian/loans?status=AWAITING_PICKUP POST /librarian/loans/confirm-pickup	View AWAITING_PICKUP orders with codes and 24 h deadlines for reference; enter pickup code to confirm handover when reader arrives at the counter → loan BORROWED + due dates set per role × doc-type; can cancel order with mandatory reason.
14	Cancel Order	POST /librarian/loans/{id}/cancel	Cancel with a mandatory reason; held copies restored and the reader notified
15	Active Borrowings	GET /librarian/loans?status=BORROWED	BORROWING items with due dates / overdue markers
16	Process Return	POST /librarian/loans/details/{id}/return	Record condition, compute fine, restore copy; loan completes when all items returned & fines settled
17	Book / Stock	GET/POST	Full CRUD incl. copy

	Management	/librarian/books/**	counts
18	Collect Fine	POST /librarian/loans/details/{id}/pay-fine	Mark an overdue fine collected; completes the loan when fully settled
19	Borrowing-Rule & Visibility Config	GET/POST /librarian/rules	Edit quotas/durations/renewals and toggle document-type visibility per role
20	In-House Reading Slip	GET/POST /librarian/inhouse	Issue a read-on-site slip for a RESTRICTED document by reader code
21	Batch Return	POST /librarian/loans/{loanId}/return-batch	Return multiple books in a single request; each item records condition and computes its fine independently.
22	Book Batch Import	GET /librarian/books/new-batch POST /librarian/books/batch	Import books from an Excel file with per-row validation; returns a JSON result before committing.
23	Book Excel Template	GET /librarian/books/import-template	Download the .xlsx template used for the batch book-import form.

Administrator features (users & system)

#	Feature	Route	Description
24	User Management	GET /admin/users	List/search users; lock/unlock cards; reset password; delete
25	Create User	POST /admin/users	Create accounts (incl. LIBRARIAN) and assign role
26	Permission Matrix	GET & POST /admin/permissions	Grant/revoke functional permissions per role (book CRUD, return, pay-fine, cancel, in-

			house, rule-manage, borrow/renew/self-cancel)
27	Login & Routing	GET/POST /login (+ GET /)	Form login; role-based redirect
28	Batch Create Users	POST /admin/users/batch	Import user accounts from an Excel file; returns per-row validation results.
29	Import Preview	POST /admin/users/import-preview	Parse and preview an uploaded Excel file before committing the user import.
30	Download User Template	GET /admin/users/import-template	Download the .xlsx template used for the batch user-import form.

6.2 Key Business Logic Components

RuleEngine.java (util/)

Method	Description
validate(User, List<CartItem>)	Account/cart/type/quota checks incl. visibility; returns ValidationResult.
calculateDueDate(role, docType, start)	Adds borrow_days; returns null for unlimited.
canRenew(detail, rule)	renewal_count < max_renewals and max_renewals > 0.
calculateFine(detail)	ChronoUnit.DAYS between due date and return date × 2,000 VND.

LoanService.java (service/)

Method	Description
submitLoan(user, cart)	Validate, verify stock, create AWAITING_PICKUP loan + pickup code + 24h deadline, hold copies.

confirmPickupByLibrarian(code)	Librarian confirms handover: finds AWAITING_PICKUP loan by pickup code → BORROWED + due dates set per role × doc-type; notifies reader.
cancelLoan(loanId, reason, byRole)	Librarian cancels with a mandatory reason; restores copies, records who cancelled, notifies the reader.
cancelExpiredPickups()	Cancel all AWAITING_PICKUP loans past their deadline (called by scheduler).
returnBook(detailId, condition)	Record return & condition, compute the fine, restore the copy; the loan reaches COMPLETED once all items are returned and all fines are settled.
renewBook(detailId, user)	Ownership + BORROWING + renewal-limit checks, then extend due date.
cancelByReader(loanId, user)	Reader cancels an own AWAITING_PICKUP order; restores the held copies.
payFine(loanDetailId)	Mark an item's overdue fine as collected; completes the loan when every item is settled.
lendInHouse(reader, bookId)	Create an in-house (read-on-site) BORROWED loan for a RESTRICTED document, due same day.
sendDueReminders()	Push due-soon / overdue notifications to readers (implemented; not yet scheduled).

RuleService.java (service/)

Method	Description
findAllOrdered()	All rules ordered by role & type for the config page.
visibleDocTypes(role)	Set of doc types where visible = true for that role.

updateRules(edits)

Persist max_quantity / borrow_days / renewals / visibility edits.

UserService also provides create(...) for admin-created accounts and changePassword(...) for self-service password changes (both using BCrypt).

NotificationService.java (service/) creates in-app notifications for business events via notify(user, ...) and notifyRole(role, ...), and exposes unreadCount / recent / findAll / markRead / markAllRead consumed by the navbar bell and the /notifications page.

PermissionService.java (service/) reads and writes the role $\times$ permission matrix (has, grantedFor, updateGrantsForRole); LibrarianController calls has(...) to gate each operation and the templates use it to show or hide action buttons.

6.3 Changes & Fixes Applied in This Version

Item	Type	Description
Self-service borrowing	Feature	Removed approval; loans are granted immediately with a pickup code and a 24h window.
Librarian confirms pickup by code	Feature	Librarian enters the reader's pickup code at /librarian/loans/confirm-pickup to confirm handover; replaces manual paper ledger entry.
Auto-cancel job	Feature	Scheduled task releases held copies if a reader does not collect within 24h.
Admin/Librarian separation	Security	Disjoint roles: the admin handles user accounts + the functional-permission matrix, the librarian handles operations + borrowing-rule configuration.
On-screen	Feature	borrowing_rules gains a

configuration		'visible' flag; the librarian edits rules & visibility on /librarian/rules, while the admin grants functional permissions on /admin/permissions.
Overdue fine bug	Bug fix	calculateFine used Period.getDays() (day-of-month only). Now uses ChronoUnit.DAYS.between for the true day count.
Stock-hold bug	Bug fix	submitLoan now verifies availability for all items before any decrement, preventing leaked held copies.
Carried-over v2 fixes	Bug fix	Transactional cancelLoan, renewal ownership check, and BORROWING-status guard remain in place.
Default-password account seed	Feature	data.sql seeds 82 accounts (2 admins + 20 per role) with default password password123; no manual ids so auto-increment stays correct for admin-created users.
Self-service password change	Feature	Any authenticated user changes their password at /account/password (UserService.changePassword + AccountController + change-password.html).
In-app notifications	Feature	NotificationService raises in-app alerts for loan / pickup / return / fine / cancel / completion events; navbar bell + /notifications page.
In-house reading	Feature	Librarian issues read-on-site

		slips for RESTRICTED (internal) documents via /librarian/inhouse.
Fine settlement & COMPLETED	Feature	Per-item fine_paid flag; a loan reaches COMPLETED only after all items are returned and all fines settled.
Functional-permission matrix	Security	Permission enum + role_permissions table + PermissionService gate every librarian action and toggle UI buttons.
Reader self-cancel	Feature	A reader can cancel an own AWAITING_PICKUP order, releasing the held copies.

7. Task Assignment

The work is split into four vertical slices (entity → repository → service → controller → view), updated to cover the new self-service and configuration features.

Member	Module	Key Files
Phạm Minh Tuyên	Auth, Users & Permissions (+ account seeding & password change)	SecurityConfig, AuthController, UserService, AdminController (users/permissions), PermissionService, RolePermission/Permission, users.html, permissions.html, AccountController, change-password.html, data.sql (account & permission seed)
Lê Hà Hải Vân	Catalogue & Book Search	BookService, BookRepository, Book.java, home.html,

		book-detail.html, books.html, book-form.html
Bùi Thanh Thảo	Borrowing Rules, Visibility & Cart	RuleEngine, RuleService, BorrowingRule.java (+visible), CartItem, RuleConfigForm, cart.html, rules.html, data.sql (rules seed)
Nguyễn Thị Thu Hà	Loan Lifecycle, Pickup & Scheduling	LoanService, NotificationService, Loan.java (pickup/in-house/cancel fields), LoanDetail.java (fine_paid), PickupExpiryScheduler, LibrarianController (loans/returns/inhouse), Notification.java, loans.html, my-loans.html, inhouse.html, notifications.html

8. Installation & Setup Guide

8.1 Prerequisites

Tool	Version	Purpose
JDK	21+	Compile & run
Apache Maven	3.8+	Build & dependencies
Web browser	Any modern	Access the app
MySQL	8.0+ (optional)	Production only

8.2 Run in Development (H2)

Step 1: Clone the repository

```
git clone https://github.com/tuyenpham2409/library-management.git
```

```
cd library-management
```

Step 2: Verify JDK installation

```
java -version
```

```
mvn -version
```

Step 3: Build & run the application

```
mvn spring-boot:run
```

Step 4: Access the application

- Main app: <http://localhost:8080>

+ User and password have been set up in the interface

- H2 console: <http://localhost:8080/h2-console>

+ JDBC URL: jdbc:h2:mem:librarydb

+ User: sa

+ Password: (blank)

8.3 Sample Accounts (password: password123)

On startup the system seeds 82 accounts: 2 administrators (admin, admin02) and 20 each of student (student01-20), lecturer (lecturer01-20), researcher (researcher01-20) and librarian (librarian01-20). All share the default password password123 and should change it after first login at /account/password. Representative accounts:

Login Code	Role	Card Status	Notes
admin	ADMIN	ACTIVE	Users + functional-permission matrix only
librarian01	LIBRARIAN	ACTIVE	Operations: pickup orders, returns & fines, book/stock, borrowing rules, in-house slips

student01	STUDENT	ACTIVE	Normal student
student02	STUDENT	ACTIVE	Lock it from the admin page to test locked-account validation
lecturer01	LECTURER	ACTIVE	Unlimited duration on textbooks
researcher01	RESEARCHER	ACTIVE	Extended specialised-reference quotas

8.4 How to Test the New Flow

1. Sign in as student01, add a textbook, submit — note the pickup code shown.
2. Sign in as librarian01; go to `/librarian/loans?status=AWAITING_PICKUP`; enter the pickup code noted in Step 1 → the loan becomes Borrowed with due dates set per role × document type.
3. Sign in as librarian01 to see pickup orders and process returns; admin pages are blocked (403).
4. Sign in as librarian01 → `/librarian/rules` → toggle a document type's visibility off for STUDENT → that type disappears for students. Then sign in as admin → `/admin/permissions` → revoke a permission for LIBRARIAN and confirm the matching action is blocked.
5. To test auto-cancel quickly, lower the scheduler `fixedRate` or set `pickup_deadline` to the past in the H2 console.

9. Testing & Validation

9.1 Functional Test Scenarios (verified)

ID	Scenario	Expected Result	Result
T-01	Self-service borrow	Loan created AWAITING_PICKUP with a pickup code; copy held; no approval	Pass

T-02	Librarian enters invalid pickup code	Error message: no Awaiting_Pickup loan found for that code	Pass
T-03	Librarian enters valid pickup code	Loan → Borrowed; due dates set per role × doc-type; reader notified via in-app notification	Pass
T-04	Reader cannot access librarian routes	GET /librarian/dashboard → 403 Forbidden for STUDENT role	Pass
T-05	Role separation — Admin	/admin/users & /admin/permissions = 200; /librarian/** = 403	Pass
T-06	Role separation — Librarian	/librarian/dashboard, /loans, /books, /rules = 200; /admin/** = 403	Pass
T-07	Login routing	admin → /admin/users; librarian → /librarian/dashboard	Pass
T-08	Hide doc type for STUDENT	Type disappears from catalogue & add-to-cart blocked	Pass
T-09	Re-show doc type	Type reappears for the role	Pass
T-10	Edit quota on UI	New max quantity enforced at submit	Pass
T-11	Return & overdue fine	Fine = overdue days × 2,000 VND (full day count)	Pass
T-12	Cancel restores stock	Copies returned to catalogue	Pass
T-13	In-house reading	Librarian issues an in-	Pass

	slip	house slip for a RESTRICTED document; copy held, due same day	
T-14	Fine collection & completion	After a return with an overdue fine the loan stays BORROWED until pay-fine, then becomes COMPLETED	Pass
T-15	Reader self-cancel	Reader cancels an own AWAITING_PICKUP order; held copies restored	Pass
T-16	In-app notification	Creating / confirming / cancelling a loan produces the matching navbar notification	Pass
T-17	Permission gate	Revoking BOOK_DELETE for LIBRARIAN blocks the delete action (flash error)	Pass

9.2 Build & Runtime Verification

- mvn clean package — BUILD SUCCESS on Spring Boot 3.4.13 / Java 21.
- Application starts on port 8080; data.sql seeds 82 users, 24 books, 12 borrowing rules, the default role-permission matrix and demo loans/notifications without error.
- Scheduled auto-cancel job is registered via @EnableScheduling.

10. Business Process: Flowchart & Walkthrough

This section documents the document-borrowing workflow twice: first the original counter-based process of the Phenikaa University Library (regulation

QT.03.TV.DV), then the digital self-service process implemented by our system, followed by a concise side-by-side comparison.

10.1 Original Process — Phenikaa University Library (QT.03.TV.DV)

Service hours: Monday–Friday, 08:00–11:30 and 13:30–16:30. The counter-based borrowing procedure has eight steps. The flowchart below reproduces the official regulation; the responsible party and the associated forms are listed per step.

No.	Flowchart (step)	Performed by	Coordinates with	Form
1	Present card	Reader	Library staff	HD.01.TV.DV.02
2	Search the catalogue	Reader	Library staff	HD.01.TV.DV.01; BM.03.QT.03.TV.DV
3	Fill in the request slip	Reader	Library staff	BM.01.QT.03.TV.DV (or record the loan directly in Koha)
4	Retrieve from the stacks	Library staff	—	HD.01.TV.DV.01
5	Complete the loan	Library staff	—	BM.01.QT.03.TV.DV; Koha software
6	Return at the staff desk	Reader	Library staff	BM.02.QT.03.TV.DV; Koha software
7	Process return / renewal / handle violations (if any)	Library staff	—	BM.02.QT.03.TV.DV; HD.01.TV.DV.02
8	Reshelve / handle documents per violation status	Library staff	—	HD.01.TV.DV.04; QT.01.TV.DV.04; QT.01.TV.DV.05

Walkthrough:

- Step 1 — Present card: The reader comes to the borrowing room and presents their card; library staff check that the card is valid before lending any document.
- Step 2 — Search the catalogue: The reader searches on a library lookup terminal or at elib.phenikaa-uni.edu.vn / library.phenikaa-uni.edu.vn (by title, author, keyword, ISBN, classification code or course code; advanced search and

Z39.50/SRU lookup are available), then views the catalogue record in detail (accession number, classification, publication data, etc.).

- Step 3 — Fill in the request slip: Complete the document request slip BM.01.QT.03.TV.DV and bring it to the librarian's desk (Room P104 – Building A4), or record the loan directly in the Koha software.
- Step 4 — Retrieve from the stacks: Library staff go into the stacks and pick the documents listed on the reader's request slip (the stacks are arranged per HD.01.TV.NV.04).
- Step 5 — Complete the loan: Library staff enter the data into the software's lending module / the borrow–return ledger (BM.02.QT.03.TV.DV); the reader checks the documents, takes the card back and signs. Quotas and loan durations apply per reader type (student / lecturer / researcher) as specified in §3.2.
- Step 6 — Return documents: The reader returns documents at Room P104, Building A4. Renewal: bring the document and card to the counter; renewal is only allowed if the document is still 'available' and has no pending reservation. Overdue: library staff compile an overdue list and send a notice to the reader's unit (BM.04 & BM.05.QT.03.TV.DV).
- Step 7 — Process the return: Library staff inspect the document on return, record the return in the software / have the reader sign the ledger; if it is damaged, handle it under QT.05.TV.DV.
- Step 8 — Reshelve documents: The librarian returns the document to its correct place on the shelf per HD.01.TV.DV.04.

10.2 Our System — Digital Self-Service Process

Our web system keeps the same business intent but removes the counter-approval bottleneck. Trust is enforced by a one-time pickup code: the librarian enters it to confirm handover instead of filling in paper slips. The flowchart below maps each step to its actor, the system logic involved, and the concrete route / artifact produced.

No.	Step (flowchart)	Performed by	System / Coordination	Route / Artifact
1	Log in	Reader	Spring Security (BCrypt); role-based routing	GET /login → /client/home

2	Search & view details	Reader	Shows only visible document types (visibleDocTypes)	GET /client/home, /client/books/{id}
3	Add to cart	Reader	6 checks: permission · duplicate · ≤ 5 · not restricted · visible · in stock	POST /client/cart/add(- ajax)
4	Submit loan request (self-service)	Reader	RuleEngine 3-tier validate → create AWAITING_PICKUP loan + 6- char code + hold copies (copies-1)	POST /client/loans/sub mit
5	Collect books (self-service)	Reader	Reader visits the library within 24 h, retrieves reserved books from open shelves; no system step	—
6	Librarian confirms pickup	Reader + Librarian	confirmPickupBy Librarian: finds AWAITING_PICKUP loan by pickup code → BORROWED + due_date per role × doc_type; reader notified	POST /librarian/loans/c onfirm-pickup
7	Renew (if allowed)	Reader	canRenew: renewalCount < maxRenewals; +renewalDays	POST /client/loans/rene w/{detailId}
8	Return & collect fines	Librarian	returnBook: fine = overdue days × 2,000 VND, restore copies;	POST /librarian/loans/d etails/{id}/return , .../pay-fine

			pay-fine; loan → COMPLETED when all returned & all fines paid	
9	Background jobs (automatic)	System	Every 10 min cancels orders not collected within 24h (restores copies); reminds of due-soon/overdue items	PickupExpiryScheduler; LoanService.sendDueReminders()
10	Order branches	Reader / Librarian / System	Self-cancel (before pickup) · librarian cancel with reason · scheduler auto-cancel — all restore copies. Internal documents: issue an in-house reading slip	/client/loans/{id}/cancel; /librarian/loans/{id}/cancel; /librarian/inhouse

Walkthrough:

- Step 1 — Log in: Authentication uses the student/staff code + a BCrypt password (Spring Security). At “/”, the system routes: Admin → account/permission management; Librarian → the operations dashboard; Reader (student/lecturer/researcher) → the catalogue page. Every URL is guarded by two layers: the role gate (SecurityConfig) and granular permissions (PermissionService).
- Step 2 — Search & view details: The reader searches by keyword / document type / availability; the catalogue shows only the document types the role is allowed to see (configured on the librarian's rules page).
- Step 3 — Add to cart: The cart lives in the HTTP session and passes six checks before an item is added — the BORROW permission, no duplicate in the cart, a

total of ≤ 5 items, not an internal (RESTRICTED) document, the document type is visible to the role, and the book still has available copies.

- Step 4 — Submit the loan request: The RuleEngine validates in three tiers (card not locked · no outstanding overdue document · within the per-type quota). If valid, an Awaiting_Pickup loan is created immediately, a 6-character pickup code is generated, a 24-hour pickup deadline is set and the copies are held (availableCopies decremented) — with no librarian approval.
- Step 5 — Prepare the books: The librarian views the list of awaiting-pickup orders (with codes and 24-hour deadlines) to gather the books; they may cancel an order with a reason (which is sent to the reader and restores the held copies).
- Step 6 — Librarian confirms pickup: The reader brings the pickup code to the library counter. The librarian enters the code at POST /librarian/loans/confirm-pickup. The system finds the Awaiting_Pickup loan by code, sets status to BORROWED, calculates each item's due date per role $\times$ document type, and sends an in-app notification to the reader.
- Step 7 — Renew: If renewals remain (renewalCount < maxRenewals and maxRenewals > 0), the due date is extended by renewalDays.
- Step 8 — Return & collect fines: The librarian processes each item's return, records its condition (GOOD/DAMAGED), computes the late fine = overdue days $\times$ 2,000 VND (ChronoUnit.DAYS) and restores the copy. Once every item is returned and every fine is paid (finePaid), the loan becomes COMPLETED.
- Step 9 — Background jobs: PickupExpiryScheduler runs every 10 minutes to cancel orders not collected within 24 hours and restore the copies; sendDueReminders() reminds readers of due-soon / overdue documents (implemented, not yet scheduled).
- Branches & supporting flows — A reader self-cancels (only before pickup), the librarian cancels (with a reason), or the scheduler auto-cancels; all restore the copies to stock. Internal (RESTRICTED) documents cannot be taken away — the librarian issues an in-house reading slip (due the same day). All roles: change their own password and view/mark in-app notifications.

10.3 Comparison — Manual Process vs. Our System

The table summarises how each system handles the same business concerns.

Criterion	Phenikaa process (manual / Koha)	Our system (self-service web)
Loan initiation	Reader fills a slip / staff record the loan; requires librarian action	Self-service: the reader creates the order, no approval needed
Order identity	Paper slip / borrow–return ledger	Auto-generated 6-character pickup code
Hand-over confirmation	Librarian hands over at the counter	Librarian enters pickup code (confirmPickupByLibrarian); reader receives in-app notification; no paper slip required
Stock holding	Manual when recording the loan	availableCopies decremented automatically when the order is created
Quota & due date	Written regulation; staff apply manually	RuleEngine applies them automatically by role × doc type (configured in the UI)
Pickup expiry	No such concept	Scheduler auto-cancels after 24h and restores the books
Late fine	Calculated manually	Automatic: overdue days × 2,000 VND; payment tracked (finePaid)
Renewal	At the counter; checks 'available' / reservations	Online if renewals remain (canRenew)
Overdue reminders	Staff compile a list and send an official notice (BM.04/05)	Event-driven in-app notifications; sendDueReminders (not yet scheduled)
Internal documents (red mark)	Cannot be taken away	RESTRICTED: borrowing blocked; librarian issues an in-house reading slip
Authorisation	By staff role	Two-layer RBAC: role

		gate + granular permission matrix
Trace & status	Ledgers / unit emails	A clear loan lifecycle with explicit statuses + in-app notifications

11. System Design Models (UML)

To present the design from complementary viewpoints, four UML models were selected as the most representative for this system: a Use Case diagram (who does what), a Class diagram (the static domain model behind the database), an Activity diagram (the self-service borrowing flow with its decision points), and a State Machine diagram (the rich lifecycle of a loan). Sequence and deployment diagrams were considered but omitted because the activity and class models already convey the behaviour and structure clearly at this scope.

11.1 Use Case Diagram

Actors are grouped exactly as the RBAC design dictates — Reader (Student/Lecturer/Researcher), Librarian, Administrator — plus a non-human Scheduler actor for the background jobs. There is no role inheritance: each actor sees only its own use cases. Log in and Change password apply to every authenticated role.

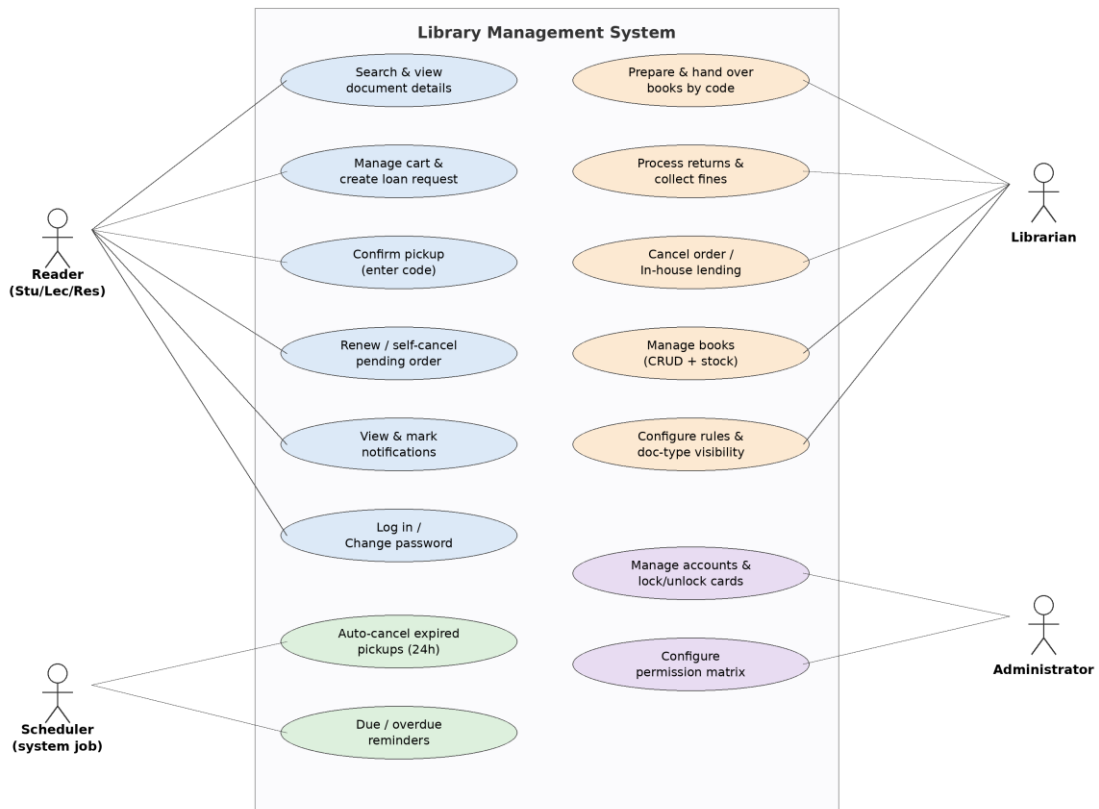


Figure 11.1 — Use Case diagram of the system.

11.2 Class Diagram — Domain Model

The static model mirrors the JPA entities and their relationships: one User has many Loans; a Loan is composed of one-or-more LoanDetails (cascade ALL); each LoanDetail references one Book; a User receives many Notifications. BorrowingRule and RolePermission are enum-keyed configuration tables that drive the rules and the permission matrix.

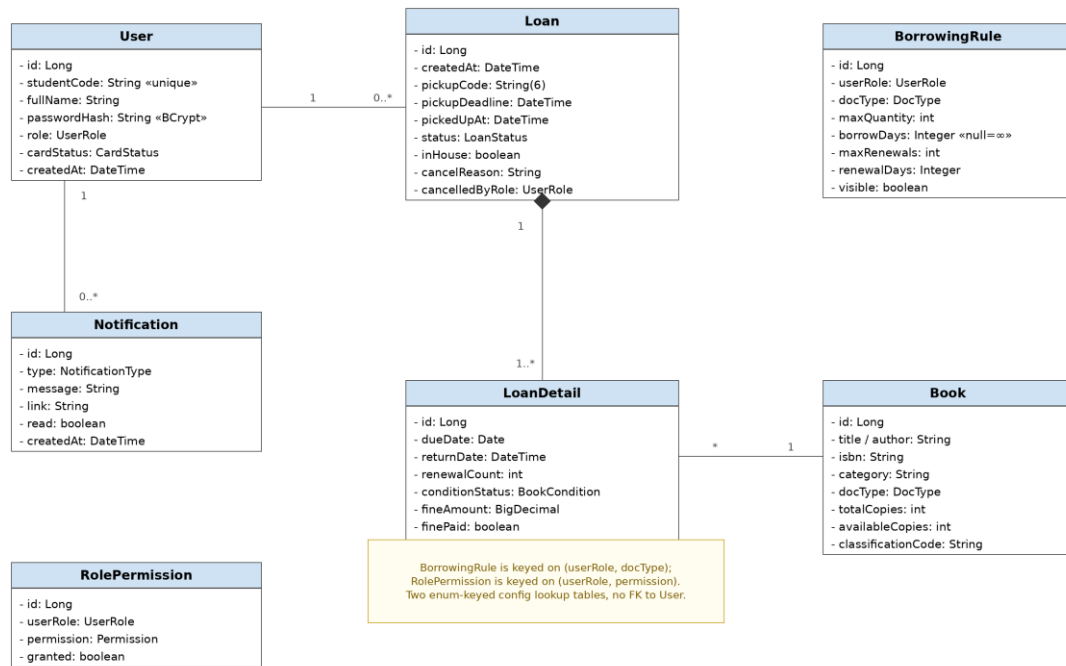


Figure 11.2 — Class diagram (domain model).

11.3 Activity Diagram — Self-Service Borrowing

The main flow runs from login through cart checks, three-tier validation, code-based pickup, renewal, return and fine settlement, to COMPLETED. Failure branches loop back to the cart or stay in the awaiting state, and the 24-hour timeout branch shows the scheduler auto-cancelling an un-collected order.

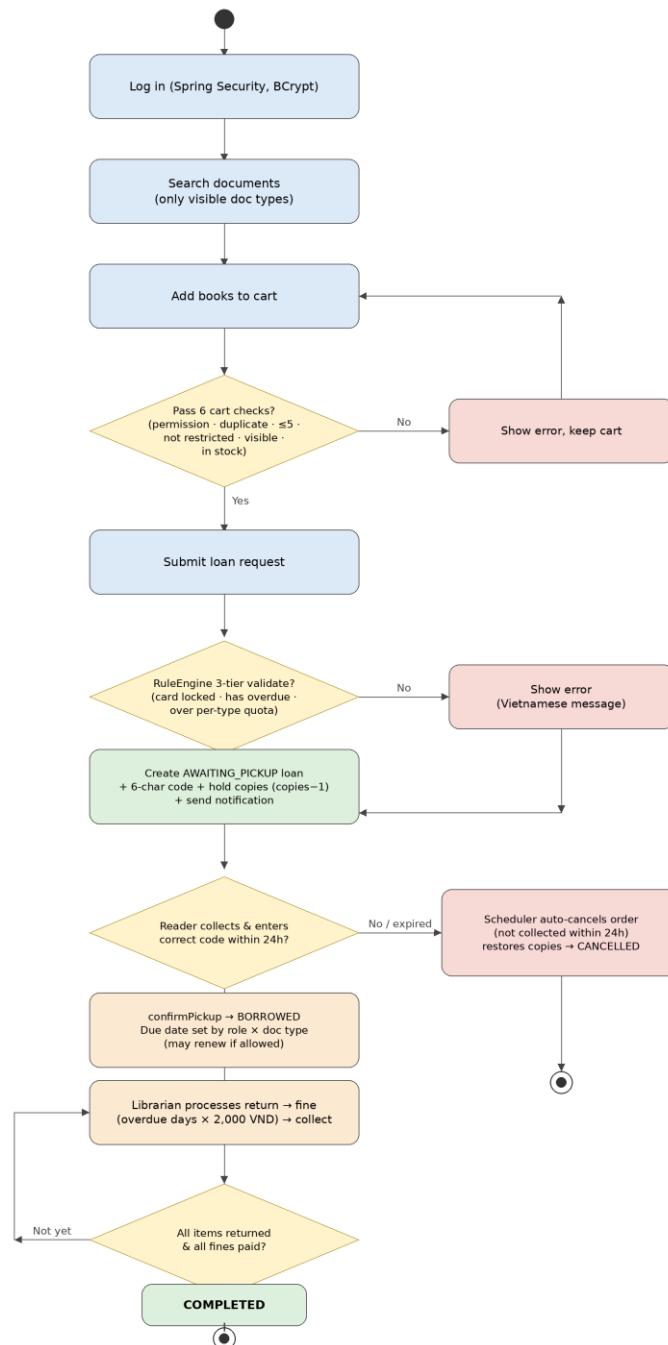


Figure 11.3 — Activity diagram: the self-service borrowing flow.

11.4 State Machine Diagram — Loan Lifecycle

A Loan starts at `AWAITING_PICKUP`, becomes `BORROWED` on a valid pickup, and reaches `COMPLETED` only after every item is returned and every fine is settled; it goes to `CANCELLED` on reader/staff cancellation or the 24-hour

timeout. The lower strip shows the LoanDetail lifecycle (RESERVED → BORROWING → RETURNED); OVERDUE is computed dynamically and is not stored as a separate state.

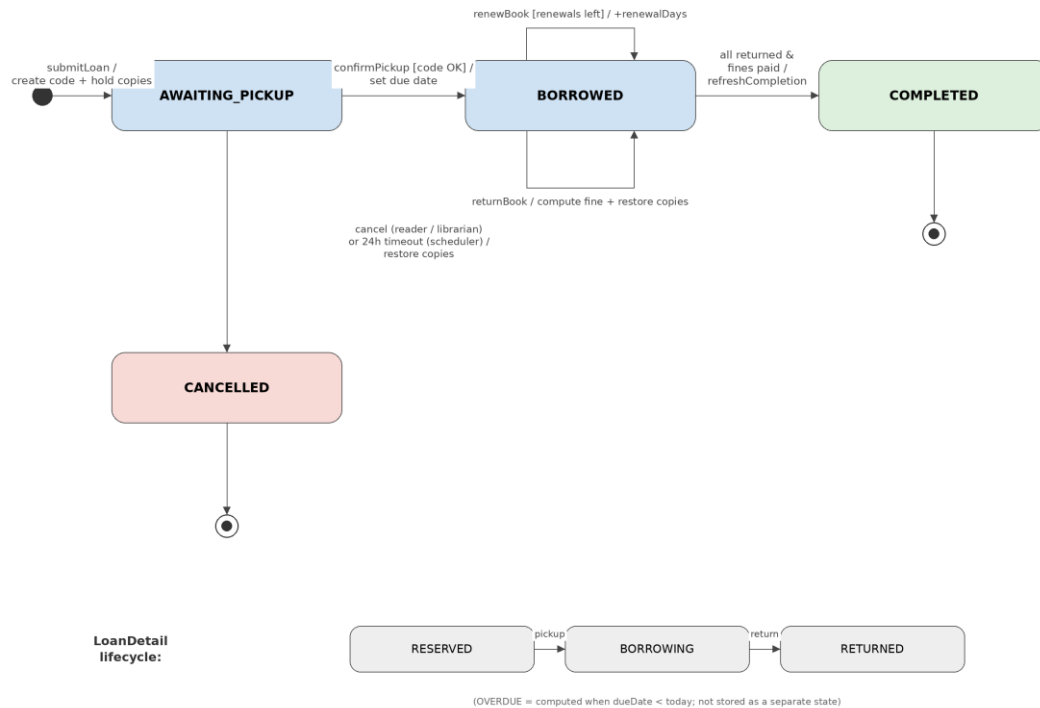


Figure 11.4 — State machine diagram: the loan lifecycle.

12. Conclusion & Future Work

12.1 Achievements

- Delivered a genuinely efficient borrowing model: no manual approval step, pickup code for librarian-confirmed handover, 24h auto-cancel window — replacing paper slips and ledger entries.
- Implemented automatic release of un-collected holds through a scheduled background job.
- Cleanly separated Administrator (user accounts + functional-permission matrix) and Librarian (operations + borrowing-rule configuration) with strict, non-inheriting RBAC.

- Added on-screen configuration — a librarian page for borrowing rules and document-type visibility, and an admin functional-permission matrix per role — requiring no code change.
- Fixed two real logic bugs (overdue-fine day count and a stock-hold leak) and upgraded to Spring Boot 3.4.13.
- Pre-seeded a default-password account base (82 accounts) and added self-service password change for every role.
- Added an in-app notification system (navbar bell + /notifications) and a librarian in-house reading workflow for restricted documents.

12.2 Limitations

- The system relies on the librarian physically verifying the reader's identity and book count before entering the pickup code.
- H2 in-memory storage loses data on restart; production needs MySQL.
- No reservation queue for currently-unavailable titles. In-app notifications are implemented, but email delivery is not; the due-soon / overdue reminder method (`LoanService.sendDueReminders`) exists but is not yet scheduled.

12.3 Future Development Roadmap

Priority	Feature	Description
High	MySQL / production deployment	Persistent storage with Flyway migrations.
Medium	Schedule & email the overdue reminders	Wire <code>LoanService.sendDueReminders()</code> to a scheduler and add email delivery (BM.04/BM.05 of QT.03.TV.DV).
Medium	Reservation queue	Reserve unavailable titles; block renewal when reserved.
Low	QR pickup code	Render the pickup code as a QR for faster counter handover.
Low	Excel / PDF reports	Export borrowing statistics and overdue lists.